

```

0000 SEPARATE (IDL) --|-----PROCEDURE-TBL_PUT --|PROCEDUR
E-TBL_PUT (-NOM_TEXTE:STRING) IS RESULT_FILE, NFILE: TEXT_IO.FILE_TYPE; RULE_LIST: SEQ_TYPE; RULE_NBR: NATURAL := 0; VOID_WAS_SEEN:
BOOLEAN := FALSE; USE INT_IO; --|-----PROCEDURE
PROCESS_RULES (-) PROCEDURE PROCESS_RULES ( CLASS_RULE: TREE ) IS ITEM_LIST: SEQ_TYPE := LIST ( CLASS_RULE ); LISTE-DES-REGLES-DEFINI
SSANT-LES-MEMBRES-DE-LA-CLASSE ITEM: TREE; RULE_NODE: TREE; VOID_SYM: TREE := STORE_SYM ( "VOID" ); --|-----
--|PROCEDURE-PUT_RULE --|PROCEDURE PUT_RULE ( RULE: TREE ) IS
SUBTYPE STR3 IS STRING ( 1..3 ); --|-----PROCED
URE-PUT_PROPER_ATTRS --|PROCEDURE PUT_PROPER_ATTRS ( TER_LIST_ARG: SEQ_TYPE; PREFIX: STR3 ) IS TER_LIST: SEQ_TYPE := TER_LIST_ARG;
G; TER: TREE; --|-----PROCEDURE-PUT
_ATTR --|PROCEDURE PUT_ATTR ( TER: TREE ) IS TER_PREFIX: STRING ( 1..3 ) := "???" ; TER_NAME: CONSTANT STRING := PRINT_NAME ( D ( X
D_SYMREP, TER ) ); BEGIN IF TER_NAME'LENGTH >= 3 THEN SI LE-NOM-EST-ASSEZ-LONG TER_PREFIX ( 1..3 ) := TER_NAME ( TER_NAME'FIRST..
TER_NAME'FIRST+2 ); EXTRAIRE LA TRANCHE DU PREFIXE END IF; IF TER_PREFIX = PREFIX SI C'EST LE PREFIXE COURANT OR ELSE (-PREFIX
= " " AND TER_PREFIX /= "as" OU SI-ON A MIS " " MAIS-QUE CE N'EST PAS UN-CONNU "AS", "LX", "SM" AND TER_PREFIX /= "LX" AND TER_PREF
IX /= "sm" ) THEN IF DI ( XD_ATTR_ID, TER ) < 0 THEN ATTRIBUT DE TYPE SEQUENCE PUT ( 'A' ); METTRE A EN DEBUT DE LIGNE ELSE
ELIF PRINT_NAME ( D ( XD_ATTR_TYPE, TER ) ) = "INTEGER" THEN TYPEAGE "INTEGER" PUT ( 'I' ); METTRE I EN DEBUT DE LIGNE ELSE
PRINT_NAME ( D ( XD_ATTR_TYPE, TER ) ) = "BOOLEAN" THEN TYPEAGE "BOOLEAN" PUT ( 'B' ); METTRE B EN DEBUT DE LIGNE ELSE
TOUT AUTRE ATTRIBUT PUT ( 'A' ); METTRE A EN DEBUT DE LIGNE END IF; PUT ( " " ); PUT ( DI ( XD_ATTR_ID, TER ), 4 );
METTRE LE N D'ATTRIBUT ( POUR LE RETROUVER DANS LA LISTE DES ATTRIBUTS ) PUT ( " " ); PUT_LINE ( PRINT_NAME ( D ( XD_SYMREP, TER ) ) & ASCII.HT & " " & PRINT_NAM
E ( D ( XD_ATTR_TYPE, TER ) ) ) ET TYPE AU FICHIER NOEUDS-COMPLETS ); END IF; BEGIN WHILE NOT IS_EMPTY (
TER_LIST ) LOOP TANT QU'IL Y A DES ATTRIBUTS PROPRES POP ( TER_LIST, TER ); PRENDRE UN ATTRIBUT IF TER.TY = DN_ATTR THEN
EN PRINCIPE L'ATTRIBUT DOIT TRE DE-CE TYPE PUT_ATTR ( TER ); L'IMPRIMER END IF; END LOOP; END PUT_PROPER_ATTRS;
PROCEDURE PUT_INHERITED_ATTRS ( CLASS_NODE: TREE; PREFIX: STR3 ) IS BEGIN IF CLASS_NODE /= TREE_VOID THEN PUT_INHERITED_ATTRS ( D ( XD
PARENT, CLASS_NODE ), PREFIX ); RECURSION AU NIVEAU SUPERIEUR PUT_PROPER_ATTRS ( LIST ( CLASS_NODE ), PREFIX ); PUIS PLACER LES-ATTRIBU
TS-PROPRES HERITES CE NIVEAU END IF; END;
BEGIN PUT ( "N " ); LIGNE-NOEUD ( PARTIE GAUCHE DE LA REGLE ) INT_IO.PUT ( RULE_NBR, 3 ); D'ABORD LE N DE
REGLE DI ( XD_NODE_ID, RULE, RULE_NBR ); LE MEMORISER DANS LA REGLE PUT_LINE ( ' ' & PRINT_NAME ( D ( XD_SYMREP, RULE ) ) );
PUIS LE NOM DE LA REGLE ( DU NOEUD ) PUT_LINE ( NFILE, PRINT_NAME ( D ( XD_SYMREP, RULE ) ) & ASCII.HT & " " ); NOM DU NOEUD AU-FICHIER N
OEUDS DECLARE PARENT: TREE := D ( XD_PARENT, RULE ); CLASSE DANS LAQUELLE EST COMPRIS LE-NOEUD DE LA REGLE BEGIN
0030 PUT_INHERITED_ATTRS (-PARENT, "as" ); METTRE LES ATTRIBUTS HERITES DU GENRE "AS" ( SYNTAXIQUES ) PUT_PROPER_ATTRS ( LIST ( RULE ), "a
s" ); IMPRIMER LA LISTE DES ATTRIBUTS PROPRES DE LA-REGLE ( LES CHAMPS DU NOEUD ) PUT_INHERITED_ATTRS (-PARENT, "LX" ); REFAI
RE ENSUITE-POUR LES "LX" ( LEXICAUX ) PUT_PROPER_ATTRS ( LIST ( RULE ), "LX" ); PUT_INHERITED_ATTRS (-PARENT, "sm" ); REFAI
E ENSUITE-POUR LES "SM" ( SEMANTIQUES ) PUT_PROPER_ATTRS ( LIST ( RULE ), "sm" ); PUT_INHERITED_ATTRS ( PARENT, " " ); PUT_P
ROPER_ATTRS ( LIST ( RULE ), " " ); END; RULE_NBR := RULE_NBR + 1; PUT_LINE ( NFILE, ASCII.HT & ' ' ); END PUT_RULE; BEGIN
PUT_LINE ( "C " & PRINT_NAME ( D ( XD_SYMREP, CLASS_RULE ) ) ); LIGNE-DEBUT DE CLASSE WHILE NOT IS_EMPTY ( ITEM_LIST ) LOOP POP
( ITEM_LIST, ITEM ); PRENDRE UN MEMBRE DE LA CLASSE TRAITEE IF ITEM.TY /= DN_ATTR THEN SI CE-N'EST PAS UN ATTRIBUT ( UN MEM
BRE ) RULE_NODE := D ( XD_CLASS_NODE, ITEM ); PRENDRE SA REGLE DE DEFINITION IF DB ( XD_IS_CLASS, RULE_NODE ) THEN LA
REGLE DEFINIT UN MEMBRE CLASSE-PROCESS_RULES ( D ( XD_CLASS_NODE, ITEM ) ); ELSE LA REGLE DEFINIT DES ATTRIBUTS IF D ( XD_SYMRE
P, RULE_NODE ) /= VOID_SYM THEN PAS LA REGLE DEFINISSANT "VOID" PUT_RULE ( D ( XD_CLASS_NODE, ITEM ) ); IMPRIMER LA REGLE NAA-ELS
0040 A VU CELLE-CI PUT_RULE ( D ( XD_CLASS_NODE, ITEM ) ); IMPRIMER LA REGLE-END IF; END IF; ELSE S'IL Y A UN TERMI
NAL C'EST UNE PROPRIETE DE CLASSE NULL; NE RIEN IMPRIMER : LA PROPRIETE SERA COLLECTEE PAR PUT_INHERITED_ATTRS COMME ATTRIBUT-HERI
TE END IF; END LOOP; PUT_LINE ( "E " & PRINT_NAME ( D ( XD_SYMREP, CLASS_RULE ) ) ); LIGNE-FIN DE CLASSE END PROCESS_RULES;
BEGIN OPEN_IDL_TREE_FILE ( NOM_TEXTE & ".lar" ); FICHIER D'ARBRE-IDL CREATE ( RESULT_FILE, OUT_FILE, NOM_TEXTE & ".tbl" ); FICHI
ER DE SORTIE .TBL CREATE ( NFILE, OUT_FILE, NOM_TEXTE & "NODES.txt" ); FICHIER DES NOEUDS COMPLETS TEXT_IO.SET_OUTPUT ( RESULT_FILE );
RULE_LIST := LIST ( STORE_SYM ( "STANDARD_IDL" ) ); LISTE-DES-REGLES DONT LA PARTIE GAUCHE EST DANS LA CATEGORIE STANDARD_IDL IF NOT IS_E
MPTY ( RULE_LIST ) THEN SI NON VIDE PROCESS_RULES ( HEAD ( RULE_LIST ) ); TRAITER END IF; RULE_LIST := LIST ( STORE_SYM ( "
ALL_SOURCE" ) ); PAREIL POUR LA CATEGORIE ALL_SOURCE IF NOT IS_EMPTY ( RULE_LIST ) THEN PROCESS_RULES ( HEAD ( RULE_LIST ) ); END IF;
RULE_LIST := LIST ( STORE_SYM ( "TYPE_SPEC" ) ); PAREIL POUR LA CATEGORIE TYPE_SPEC IF NOT IS_EMPTY ( RULE_LIST ) THEN PROCESS_RULES (
0050 HEAD ( RULE_LIST ) ); END IF; RULE_LIST := LIST ( STORE_SYM ( "NON_DIANA" ) ); PAREIL POUR LA CATEGORIE NON_DIANA IF NOT IS_EMPTY ( RULE
LIST ) THEN PROCESS_RULES ( HEAD ( RULE_LIST ) ); END IF; TEXT_IO.SET_OUTPUT ( TEXT_IO.STANDARD_OUTPUT ); CLOSE ( NFILE ); CLOSE ( RESUL
T_FILE ); CLOSE_IDL_TREE_FILE; PUT_LINE ( "OK" ); NEW_LINE; EXCEPTION WHEN NAME_ERROR => PUT_LINE ( "LE FICHIER : " & NOM_TEXTE & ".lar
EST INTROUVABLE" ); WHEN OTHERS => CLOSE ( RESULT_FILE ); CLOSE_IDL_TREE_FILE; PUT_LINE ( "ERREUR TBL_PUT" ); END TBL_PUT;

```